

AI ASSISTANT CODING

ASSIGNMENT-18.2

Name: S.Kirthan

Enrollment no: 2303A52157

Batch: 34

Semester: VI

Branch: CSE(AIML)

Lab 18 – API Integration: Connecting to External Services with Error Handling

Task Description – 1 (Air Quality API Integration)

- **Task:**

Use AI assistance to develop a Python program that retrieves real-time air quality information for a specified city.

Instructions:

- Generate Python code using the requests library.
- Handle errors such as invalid city names, missing or incorrect API keys, and network timeouts.
- Extract and display AQI value, pollution level, and dominant pollutant in a readable format.

Sample Code:

```
import requests  
  
city = "Delhi"  
  
url = "https://api.example.com/airquality"
```

Expected Output:

- A Python script that successfully fetches and displays air quality data with proper error handling.

```
import requests

def get_air_quality(city, api_key):
    try:
        # API endpoint
        url = f"https://api.waqi.info/feed/{city}?token={api_key}"

        # Send request with timeout
        response = requests.get(url, timeout=5)

        # Raise HTTP errors (like 404, 500)
        response.raise_for_status()

        data = response.json()

        # Handle API-level errors
        if data["status"] != "OK":
            print(f"Error: Invalid city name or API key.")
            return

        # Extract required data
        aqi = data["data"]["aqi"]
        dominant_pollutant = data["data"]["dominantpol"]

        # Determine pollution level
        if aqi <= 50:
            level = "Good 🟢"
        elif aqi <= 100:
            level = "Moderate 🟡"
        elif aqi <= 150:
            level = "Unhealthy for Sensitive Groups 🟠"
        elif aqi <= 200:
            level = "Unhealthy 🔴"
        elif aqi <= 300:
            level = "Very Unhealthy ⚠️"
        else:
            level = "Hazardous 🚨"

        # Display output
        print("\n🌬️ Air Quality Report")
        print(f"City: {city}")
        print(f"AQI Value: {aqi}")
        print(f"Pollution Level: {level}")
        print(f"Dominant Pollutant: {dominant_pollutant.upper()}")

    except requests.exceptions.Timeout:
        print(f"⌚ Error: Request timed out. Please try again.")
    except requests.exceptions.ConnectionError:
        print(f"🌐 Error: Network problem. Check your internet connection.")
    except requests.exceptions.HTTPError as e:
        print(f"🔥 HTTP Error: {e}")
    except Exception as e:
        print(f"❌ Unexpected Error: {e}")

# Main Program
if __name__ == "__main__":
    city = input("Enter city name: ")
    api_key = input("Enter your API key: ")

    get_air_quality(city, api_key)
```

Enter city name: Delhi
Enter your API key: 939f0f6d86108c48211aaf6af0b35ca8cf996726

🌬️ Air Quality Report
City: Delhi
AQI Value: 192
Pollution Level: Unhealthy
Dominant Pollutant: PM25

Task Description – 2 (Language Translation API)

- Task:

Integrate a Language Translation API to translate text from English to two other languages.

Instructions:

- Use AI to generate API request code for translation.
- Validate user input text and target language codes.
- Handle API response errors and service unavailability.

- Display original and translated text clearly.

Sample Code:

```
import requests
```

```
text = "Welcome to AI Assisted Coding"
```

```
target_lang = "fr"
```

Expected Output:

- A working Python script that translates user input text with graceful error handling.

The image displays two screenshots of a Jupyter Notebook interface, showing a Python script for text translation with error handling.

Top Screenshot: The notebook is titled "Lab 18(Task 2).ipynb". The code cell shows the following Python code:

```
import requests

# Supported languages
VALID_LANGUAGES = {
    "fr": "French",
    "es": "Spanish",
    "de": "German",
    "hi": "Hindi",
    "te": "Telugu"
}

def translate_text(text, target_lang):
    try:
        if not text.strip():
            return "❌ Error: Text cannot be empty."

        if target_lang not in VALID_LANGUAGES:
            return "❌ Error: Unsupported language '{target_lang}'. Supported: {list(VALID_LANGUAGES.keys())}"
        url = "https://translate.googleapis.com/translate"

        payload = {
            "q": text,
            "source": "en",
            "target": target_lang,
            "format": "text"
        }

        response = requests.post(url, data=payload, timeout=10)
        if response.status_code != 200:
            return "❌ API Error: Status Code {response.status_code}"

        try:
            data = response.json()
        except:
            return "❌ Invalid API response: {response.text}"
        if "translatedText" not in data:
            return "❌ Error: Unexpected API response format."

    except requests.exceptions.Timeout:
        return "❌ Error: Request timed out."
    except requests.exceptions.ConnectionError:
        return "❌ Error: Unable to connect to API."
    except Exception as e:
        return "❌ Unexpected Error: {str(e)}"
```

Bottom Screenshot: The notebook shows the same code cell, but with additional code for a main program and the output of the script. The main program code is:

```
# MAIN PROGRAM
if __name__ == "__main__":
    print("🔵 Language Translation Tool")
    print("Supported Languages:", VALID_LANGUAGES)

    text = input("\nEnter text in English: ").strip()

    lang1 = input("Enter first target language code: ").strip().lower()
    lang2 = input("Enter second target language code: ").strip().lower()

    print("\n• Original Text:", text)

    result1 = translate_text(text, lang1)
    result2 = translate_text(text, lang2)

    print("\n• Translated to {VALID_LANGUAGES.get(lang1, lang1)}: {result1}")
    print("\n• Translated to {VALID_LANGUAGES.get(lang2, lang2)}: {result2}")
```

The output of the script is displayed below the code cell:

```
🔵 Language Translation Tool
Supported Languages: {'fr': 'French', 'es': 'Spanish', 'de': 'German', 'hi': 'Hindi', 'te': 'Telugu'}

Enter text in English: Welcome to AI Assisted Coding
Enter first target language code: fr
Enter second target language code: hi

• Original Text: Welcome to AI Assisted Coding
• Translated to French: ❌ Error: Unable to connect to API.
• Translated to Hindi: ❌ Error: Unable to connect to API.
```

Task Description – 3 (Public Book Information API)

- Task:

Connect to a public Books API to fetch information about a book using its title or ISBN.

Instructions:

- Prompt AI to write Python code for sending GET requests to the API.
- Handle errors such as book not found, invalid input, and rate limits.
- Display book title, author, publisher, and publication year.

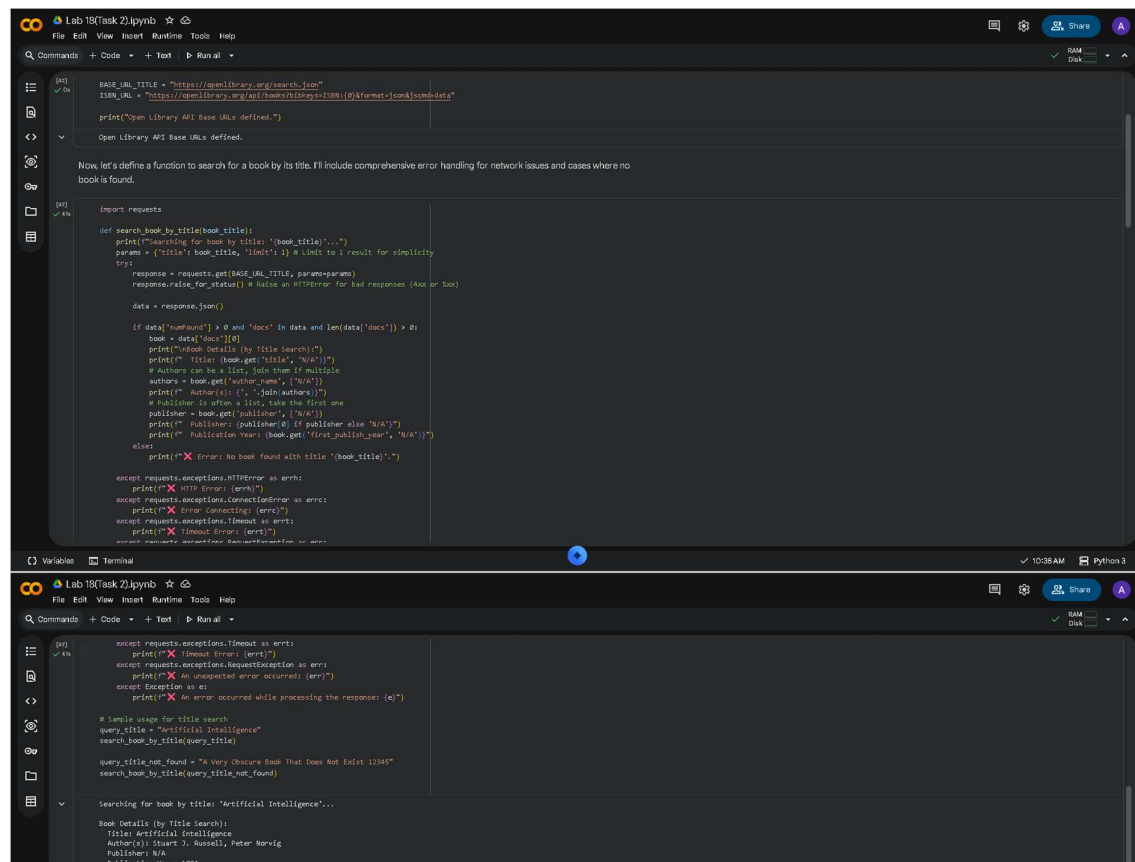
Sample Code:

```
import requests
```

```
query = "Artificial Intelligence"
```

Expected Output:

- A Python program that retrieves and displays book details with robust error handling.



```
BASE_URL = "https://openlibrary.org/search.json"
ISBN_URL = "https://openlibrary.org/api/books?bibkeys=ISBN:(0)&format=json&jsondata="

print("Open Library API Base URLs defined.")

Open Library API Base URLs defined.

Now, let's define a function to search for a book by its title. I'll include comprehensive error handling for network issues and cases where no book is found.

import requests

def search_book_by_title(book_title):
    print(f"Searching for book by title: '{book_title}'...")
    params = {'title': book_title, 'limit': 1} # limit to 1 result for simplicity
    try:
        response = requests.get(BASE_URL, params=params)
        response.raise_for_status() # Raise an HTTPError for bad responses (400 or 500)
        data = response.json()

        if data['found'] > 0 and 'docs' in data and len(data['docs']) > 0:
            book = data['docs'][0]
            print(f"Book Details (by Title Search):")
            print(f"Title: {book.get('title', 'N/A')}")
            # authors can be a list, join them if multiple
            authors = book.get('author_name', [])
            print(f"Author(s): {' '.join(authors)}")
            # publisher is often a list, take the first one
            publisher = book.get('publisher', ['N/A'])
            print(f"Publisher: {publisher[0] if publisher else 'N/A'}")
            print(f"Publication Year: {book.get('first_publish_year', 'N/A')}")
        else:
            print(f"Error: No book found with title '{book_title}'.")

    except requests.exceptions.HTTPError as errh:
        print(f"HTTP Error: {errh}")
    except requests.exceptions.ConnectionError as errc:
        print(f"Error Connecting: {errc}")
    except requests.exceptions.Timeout as errt:
        print(f"Timeout Error: {errt}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

# Sample usage for title search
query_title = "Artificial Intelligence"
search_book_by_title(query_title)

query_title_not_found = "A Very Obscure Book That Does Not Exist 12345"
search_book_by_title(query_title_not_found)
```

Searching for book by title: 'Artificial Intelligence'...

Book Details (by Title Search):

Title: Artificial Intelligence

Author(s): Stuart J. Russell, Peter Norvig

Publisher: N/A

Publication Year: 1994

The image displays two screenshots of a Jupyter Notebook environment, specifically 'Lab 18(Task 3).ipynb'. The top screenshot shows the code for a function named `search_book_by_isbn`. This function takes an ISBN number as input and uses the `requests` library to query an Open Library API. It includes comprehensive error handling for various HTTP status codes (400, 404, 408) and exceptions like `ConnectionError`, `TimeoutError`, and `RequestException`. The function prints out book details such as title, author(s), publisher, and publication year, or an error message if the book is not found.

```
import requests

def search_book_by_isbn(isbn_number):
    print(f"Searching for book by ISBN: '{isbn_number}'...")
    url = f"https://openlibrary.org/api/books?bibkeys=ISBN:{isbn_number}&format=json&limit=1&offset=0"
    response = requests.get(url)
    response.raise_for_status() # Raise an HTTPError for bad responses (400 or 500)

    data = response.json()

    # Open Library API returns a dictionary where the key is the bibkey (e.g., "ISBN:9780211765723")
    # Check if the ISBN key exists in the response data and contains actual book information (e.g., "title")
    book_key = f"ISBN:{isbn_number}"
    if book_key in data and "title" in data[book_key]: # Changed 'data' to 'title' for correct parsing
        book_data = data[book_key]
        print(f"Book Details (by ISBN search):")
        print(f"Title: {book_data.get('title', 'N/A')}")

        authors = []
        if 'authors' in book_data:
            for author_entry in book_data['authors']:
                if 'name' in author_entry:
                    authors.append(author_entry['name'])
        print(f"Author(s): {', '.join(authors) if authors else 'N/A'}")

        publishers = book_data.get('publishers', [])
        print(f"Publishers: {publishers[0]['name'] if publishers and 'name' in publishers[0] else 'N/A'}")
        print(f"Publication Year: {book_data.get('publish_date', 'N/A')}") # 'publish_date' for ISBN search
    else:
        print(f"Error: No book found with ISBN '{isbn_number}'. The API response might be empty or invalid (data).")

    except requests.exceptions.HTTPError as errh:
        if response.status_code == 400:
            print(f"Error: Book with ISBN '{isbn_number}' not found. (HTTP 400)")
        else:
            print(f"HTTP Error: {errh}")
    except requests.exceptions.ConnectionError as errc:
        print(f"Error connecting: {errc}")
    except requests.exceptions.Timeout as errt:
        print(f"Timeout Error: {errt}")
    except requests.exceptions.RequestException as err:
        print(f"An unexpected error occurred: {err}")

# Sample usage for ISBN search
sample_isbn_found = "0201559823" # Example: The Lord of the Rings (might vary, use a known ISBN)
sample_isbn_not_found = "9999999999999" # Example of a non-existent ISBN

search_book_by_isbn(sample_isbn_found)
print("\n---\n") # Separator for better readability
search_book_by_isbn(sample_isbn_not_found)
```

The bottom screenshot shows the output of the code execution. It displays the search results for the ISBN `0201559823`, which corresponds to the book "Concrete Mathematics" by Ronald L. Graham, Donald Knuth, and Oren Patashnik, published by Addison-Wesley in 1994. It also shows the error message for the non-existent ISBN `9999999999999`.

```
--- Searching for book by ISBN: '0201559823'...
Book Details (by ISBN search):
Title: Concrete mathematics
Author(s): Ronald L. Graham, Donald Knuth, Oren Patashnik
Publisher: Addison-Wesley
Publication Year: 1994

---
Searching for book by ISBN: '9999999999999'...
Error: No book found with ISBN '9999999999999'. The API response might be empty or invalid (data).
```

Task Description – 4 (Real-Time Application: Job Listings Aggregator)

- Scenario:

Develop a Job Listings Aggregator using a public Jobs API.

Requirements:

- Fetch the latest job postings for a given role or location.
- Handle errors such as invalid search terms, missing API keys, and request failures.
- Display job title, company name, location, and posting date in a numbered list.

- Implement a retry mechanism for failed API requests.

Sample Code:

```
import requests
```

```
keyword = "Python Developer"
```

Expected Output:

- A functional Python script that retrieves and displays job listings with proper error handling.

The first screenshot shows the initial setup of the script. It includes the installation of the 'requests' library and the definition of a function 'fetch_jobs' that uses a retry mechanism to fetch job listings from a specific API. The function 'display_jobs' is also defined to format and print the job details.

```
[106] pip install requests
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi<=2024.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2024.2.25)

[175] Import requests
Import time

BASE_URL = "https://www.arbitron.com/api/job-board-api"

def fetch_jobs(keyword, retries=3):
    keyword = keyword.lower()
    for attempt in range(retries):
        try:
            response = requests.get(BASE_URL, timeout=5)
            if response.status_code != 200:
                raise Exception("API request failed")
            data = response.json()
            jobs = []
            for job in data.get("data", []):
                title = job.get("title", "").lower()
                if any(word in title for word in keyword.split()):
                    jobs.append(job)
            return jobs[:10]
        except requests.exceptions.Timeout:
            print(f"Retry (attempt+1): Timeout...")
        except requests.exceptions.ConnectionError:
            print(f"Retry (attempt+1): Connection error...")

except Exception as e:
    print(f"Error: {e}")
    break
time.sleep(2)
return []

def display_jobs(jobs):
    if not jobs:
        print("⚠️ No jobs found. Try different keyword like 'developer', 'engineer'")
        return
    print(f"🔍 Job Listings:\n")
    for i, job in enumerate(jobs, 1):
        print(f"#{i} Job: {job.get('title', 'N/A')}")
        print(f"🏢 Company: {job.get('company_name', 'N/A')}")
        print(f"📍 Location: {job.get('location', 'N/A')}")
        print(f"📅 Posted: {job.get('created_at', 'N/A')}")
        print(f"-----")

# MAIN
keyword = input("Enter job role: ").strip()
if keyword:
    jobs = fetch_jobs(keyword)
    display_jobs(jobs)
else:
    print("❌ Invalid input")

Enter job role: Python Developer

🔍 Job Listings:
1. Software Developer In Test II - Java (m/f/d)
🏢 Company: Grapix
📍 Location: Heidelberg, Baden-Württemberg, Germany
📅 Posted: 1774878315
-----
2. Ruby on Rails Developer (m/w/d)
🏢 Company: alfinjo AG
📍 Location: Dresden
📅 Posted: 1774875689
-----
```

The second screenshot shows the execution of the script. The user enters 'Python Developer' as the job role, and the script successfully retrieves and displays a list of job listings with their details, including company names, locations, and posting dates.

Task Description – 5 Stock Market Data API Integration

- Task:

Use AI assistance to develop a Python program that retrieves real-time stock market data for a selected company.

Instructions:

- Generate Python code using the requests library to connect to a public Stock Market API.
- Validate user input for stock symbol.
- Handle common errors such as invalid symbols, API rate limits, missing API keys, and network failures.
- Extract and display stock name, current price, day high, day low, and percentage change.

Sample Code:

```
import requests
```

```
symbol = "AAPL"
```

- Expected Output:

A Python script that successfully fetches and displays stock market information with robust error handling.

```
Lab 18(Task 5).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

import requests
import time

API_KEY = "YOUR_API_KEY_HERE"
ALPHA_URL = "https://www.alphavantage.co/query"

def validate_symbol(symbol):
    return symbol.isalnum()

def fetch_from_alpha(symbol):
    if API_KEY == "YOUR_API_KEY_HERE" or not API_KEY:
        return None
    params = {
        "function": "GLOBAL_QUOTE",
        "symbol": symbol,
        "apikey": API_KEY
    }
    try:
        response = requests.get(ALPHA_URL, params=params, timeout=10)
        data = response.json()
        if "None" in data:
            print("⚠️ Alpha Vantage rate limit reached.")
            return None
        if "Error message" in data:
            print("❌ Invalid Alpha API key.")
            return None
        quote = data.get("Global Quote")
        if not quote:
            return None
        return {
            "source": "Alpha Vantage",
            "symbol": quote.get("01. symbol"),
            "price": quote.get("02. price"),
            "high": quote.get("03. high"),
            "low": quote.get("04. low"),
            "change_percent": quote.get("10. change percent")
        }
    except Exception as e:
        print("❌ Alpha error:", e)
        return None

def fetch_from_fmp(symbol):
    url = "https://financialmodelingprep.com/api/v3/quote/{symbol}/{apikey=demo}"
    try:
        response = requests.get(url, timeout=10)
        if response.status_code != 200:
            print("❌ API request failed.")
            return None
        data = response.json()
        if not data:
            print("❌ Invalid stock symbol.")
            return None
        stock = data[0]
        return {
            "source": "FMP API",
            "symbol": stock.get("symbol"),
            "price": stock.get("price"),
            "high": stock.get("high"),
            "low": stock.get("low"),
            "change_percent": str(round(stock.get("changePercentage", 0), 2)) + "%"
        }
    except requests.exceptions.Timeout:
        print("⌚ Request timed out.")
    except requests.exceptions.ConnectionError:
        print("🌐 Network error.")
    except Exception as e:
        print("❌ FMP error:", e)
        return None

def display_stock(data):
    print("📊 Stock Market Data")
    print("-----")
    print(f"Source: {data['source']}")
    print(f"Symbol: {data['symbol']}")
    print(f"Current Price: {data['price']}")
    print(f"Day High: {data['high']}")
    print(f"Day Low: {data['low']}")
    print(f"Change: {data['change_percent']}")

def main():
    symbol = input("Enter stock symbol (e.g., AAPL, TSLA): ").strip().upper()
    if not validate_symbol(symbol):
        print("❌ Invalid symbol. Use only letters/numbers.")
    else:
        data = fetch_from_alpha(symbol)
        if data:
            display_stock(data)
        else:
            data = fetch_from_fmp(symbol)
            if data:
                display_stock(data)
            else:
                print("⌚ Retrying in 2 seconds...")
                time.sleep(2)
        if attempt < retries - 1:
            print("⌚ Retrying in 2 seconds...")
            time.sleep(2)
        print("❌ All retry attempts failed.")
        print("⚠️ Possible issues:")
        print("  - No internet connection")
        print("  - API temporarily down")
        print("  - Invalid stock symbol")
        if __name__ == "__main__":
            main()
```

```
Lab 18(Task 5).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

try:
    response = requests.get(url, timeout=10)
    if response.status_code != 200:
        print("❌ API request failed.")
        return None
    data = response.json()
    if not data:
        print("❌ Invalid stock symbol.")
        return None
    stock = data[0]
    return {
        "source": "FMP API",
        "symbol": stock.get("symbol"),
        "price": stock.get("price"),
        "high": stock.get("high"),
        "low": stock.get("low"),
        "change_percent": str(round(stock.get("changePercentage", 0), 2)) + "%"
    }
except requests.exceptions.Timeout:
    print("⌚ Request timed out.")
except requests.exceptions.ConnectionError:
    print("🌐 Network error.")
except Exception as e:
    print("❌ FMP error:", e)
    return None

def display_stock(data):
    print("📊 Stock Market Data")
    print("-----")
    print(f"Source: {data['source']}")
    print(f"Symbol: {data['symbol']}")
    print(f"Current Price: {data['price']}")
    print(f"Day High: {data['high']}")
    print(f"Day Low: {data['low']}")
    print(f"Change: {data['change_percent']}")

def main():
    symbol = input("Enter stock symbol (e.g., AAPL, TSLA): ").strip().upper()
    if not validate_symbol(symbol):
        print("❌ Invalid symbol. Use only letters/numbers.")
    else:
        data = fetch_from_alpha(symbol)
        if data:
            display_stock(data)
        else:
            data = fetch_from_fmp(symbol)
            if data:
                display_stock(data)
            else:
                print("⌚ Retrying in 2 seconds...")
                time.sleep(2)
        if attempt < retries - 1:
            print("⌚ Retrying in 2 seconds...")
            time.sleep(2)
        print("❌ All retry attempts failed.")
        print("⚠️ Possible issues:")
        print("  - No internet connection")
        print("  - API temporarily down")
        print("  - Invalid stock symbol")
        if __name__ == "__main__":
            main()
```

```
Lab 18(Task 5).ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

retries = 3
for attempt in range(retries):
    print(f"\n⏱️ Attempt {attempt + 1}...")
    # Try alpha (optional)
    data = fetch_from_alpha(symbol)
    # fallback to FMP (main working API)
    if not data:
        print("⚠️ Switching to FMP API...")
        data = fetch_from_fmp(symbol)
    if data:
        display_stock(data)
        return
    else:
        if attempt < retries - 1:
            print("⌚ Retrying in 2 seconds...")
            time.sleep(2)
        print("❌ All retry attempts failed.")
        print("⚠️ Possible issues:")
        print("  - No internet connection")
        print("  - API temporarily down")
        print("  - Invalid stock symbol")
        if __name__ == "__main__":
            main()
```

Enter stock symbol (e.g., AAPL, TSLA): AAPL

```
⏱️ Attempt 1...
⚠️ Switching to FMP API...
❌ API request failed.
⌚ Retrying in 2 seconds...
⏱️ Attempt 2...
⚠️ Switching to FMP API...
❌ API request failed.
⌚ Retrying in 2 seconds...
⏱️ Attempt 3...
⚠️ Switching to FMP API...
❌ API request failed.
❌ All retry attempts failed.
⚠️ Possible issues:
  - No internet connection
  - API temporarily down
  - Invalid stock symbol
```